



Отчет по лабораторной работе

Задание 1 по курсу «Информационные технологии» вариант 11

Учебная группа

М7О-207БВ-24

Студент

Никитин П.А. _____

Преподаватель

Барчев Н.Б. _____

Содержание

1 Текст задания лабораторной работы	1
2 Псевдокод	2
3 Сведения о программной реализации	5
3.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования	5
3.2 Описание входных и выходных данных	5
3.3 Краткое описание спроектированных программных единиц	6
3.4 Список диагностических сообщений с указанием возможных причин их появления	11
4 Инструкция использования программы	13
5 Инструкция использования программы	13
6 Листинг программной разработки	17
7 Результаты тестирования	26

1

Текст задания лабораторной работы

Задание 1 по курсу «Информационные технологии» вариант 11

1. Подготовить программу, формирующую на основе информации, вводимой пользователем с клавиатуры, два внешних текстовых файла:
 - файл автомобилей: состоит из записей, каждая из которых включает три поля – марки, модели и номерного знака;
 - файл регистрационных сведений: состоит из записей, каждая из которых включает четыре поля – номерного знака, фамилии и адреса владельца, года выпуска.
2. В процессе проектирования предусмотреть необходимые по смыслу задания проверки корректности данных, а также адекватное задаче взаимодействие с пользователем. **Учесть, что ограничения на размеры файлов отсутствуют, файлы в общем случае могут иметь различные длину.**
В обязательном порядке использовать языковые средства организации программных единиц.
Не использовать программные средства, поддерживающие работу с базами данных.
3. Программу подготовить с использованием средств одной из реализаций языка программирования C++, **ввод и вывод информации реализовать при помощи средств языка программирования C.** Выполнить тестирование обеих программ согласно стратегии черного ящика, используя соответствующие критерии тестирования.

2

Псевдокод

```
1 ПОДКЛЮЧЕНИЕ СОДЕРЖИМОГО "file_handler.hpp"
2
3 ВЫЗОВ функции main()
4
5 функция main():
6     УСТАНОВИТЬ ЛОКАЛЬ: ru_RU.UTF-8
7     ВЫВЕСТИ: Заголовок программы
8     ОБЪЯВИТЬ БУФЕР[512]
9     ВЫВЕСТИ ГЛАВНОЕ МЕНЮ С ОПЦИЯМИ: 1 - автомобильные данные, 2 - регистрационные данные,
q - выход
10 ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ВВЕЛ q:
11     СЧИТАТЬ ВЫБОР ПОЛЬЗОВАТЕЛЯ
12     ЕСЛИ ВЫБОР РАВЕН '1' ИЛИ '2' ТО:
13         ВЫВЕСТИ ЗАГОЛОВОК ВЫБРАННОГО РЕЖИМА
14         ВЫЗВАТЬ write_data(БУФЕР, РАЗМЕР_БУФЕРА, ВЫБОР)
15         ОЖИДАТЬ НАЖАТИЕ ENTER ДЛЯ ПРОДОЛЖЕНИЯ
16     ИНАЧЕ ЕСЛИ ВЫБОР РАВЕН 'q' ИЛИ 'Q' ТО:
17         ПРЕРВАТЬ ЦИКЛ
18     ИНАЧЕ:
19         ВЫВЕСТИ СООБЩЕНИЕ ОБ ОШИБКЕ ВВОДА
20     ВЫВЕСТИ СООБЩЕНИЕ О ЗАВЕРШЕНИИ ПРОГРАММЫ
21     ЗАВЕРШИТЬ ПРОГРАММУ
22
23 функция file_info(БУФЕР, РАЗМЕР_БУФЕРА, ИМЯ_ПО_УМОЛЧАНИЮ):
24     ЗАПРОСИТЬ У ПОЛЬЗОВАТЕЛЯ ИМЯ ФАЙЛА
25     ПОКА НЕ ВВЕДЕНО КОРРЕКТНОЕ ИМЯ:
26         СЧИТАТЬ ВВОД
27         ЕСЛИ ВВОД РАВЕН 'q' ТО: ВЕРНУТЬ ПУСТОЕ ИМЯ
28         ЕСЛИ ВВОД ПУСТОЙ ТО: ИСПОЛЬЗОВАТЬ ИМЯ ПО УМОЛЧАНИЮ
29         ЕСЛИ ИМЯ НЕ ПРОШЛО ВАЛИДАЦИЮ ТО:
30             ВЫВЕСТИ СООБЩЕНИЕ О НЕВЕРНОМ ФОРМАТЕ
31             ПОВТОРИТЬ ЗАПРОС
32     ПРОВЕРИТЬ СУЩЕСТВОВАНИЕ ФАЙЛА
33     ЕСЛИ ФАЙЛ СУЩЕСТВУЕТ ТО:
34         ЗАПРОСИТЬ РЕЖИМ РАБОТЫ: добавить (Enter), перезаписать (N), отмена (q)
35         ЕСЛИ ВЫБРАНА ОТМЕНА ТО: ВЕРНУТЬ ПУСТОЕ ИМЯ
36         ЕСЛИ ВЫБРАНА ПЕРЕЗАПИСЬ ТО: РЕЖИМ = "w"
37         ИНАЧЕ: РЕЖИМ = "a"
38     ИНАЧЕ:
39         РЕЖИМ = "w"
40     ВЕРНУТЬ ИМЯ_ФАЙЛА И РЕЖИМ_ОТКРЫТИЯ
41
```

```
42 функция write_data(БУФЕР, РАЗМЕР_БУФЕРА, ТИП_ДАННЫХ):
43     ВЫЗВАТЬ file_info ДЛЯ ПОЛУЧЕНИЯ ИМЕНИ И РЕЖИМА ОТКРЫТИЯ ФАЙЛА
44     ЕСЛИ ИМЯ ФАЙЛА ПУСТОЕ ТО:
45         ВЕРНУТЬСЯ В ГЛАВНОЕ МЕНЮ
46     ОТКРЫТЬ ФАЙЛ С УКАЗАННЫМИ ПАРАМЕТРАМИ
47     ЕСЛИ ФАЙЛ НЕ ОТКРЫЛСЯ ТО:
48         ВЫВЕСТИ СООБЩЕНИЕ ОБ ОШИБКЕ
49         ВЕРНУТЬСЯ
50     ВЫВЕСТИ СООБЩЕНИЕ ОБ УСПЕШНОМ ОТКРЫТИИ
51     ВЫВЕСТИ ПОДСКАЗКИ: 'q' - завершить ввод, 'menu' - выход в меню
52     ОПРЕДЕЛИТЬ НАБОРЫ ПОЛЕЙ И ПОДСКАЗОК В ЗАВИСИМОСТИ ОТ ТИПА_ДАННЫХ
53     ОБНУЛИТЬ СЧЕТЧИК ЗАПИСЕЙ И НОМЕР ТЕКУЩЕГО ПОЛЯ
54     ПОКА ПОЛЬЗОВАТЕЛЬ НЕ ЗАВЕРШИЛ ВВОД:
55         ВЫВЕСТИ НАЗВАНИЕ ТЕКУЩЕГО ПОЛЯ И ПОДСКАЗКУ К НЕМУ
56         СЧИТАТЬ ВВОД ПОЛЬЗОВАТЕЛЯ
57         ЕСЛИ ВВОД РАВЕН 'q' ТО: ЗАВЕРШИТЬ ЦИКЛ
58         ЕСЛИ ВВОД РАВЕН 'menu' ТО:
59             ЗАКРЫТЬ ФАЙЛ
60             ВЕРНУТЬСЯ В ГЛАВНОЕ МЕНЮ
61         ЕСЛИ ВВОД ПУСТОЙ ТО:
62             ВЫВЕСТИ СООБЩЕНИЕ ОБ ОШИБКЕ
63             ПОВТОРИТЬ ВВОД
64         ПРОВЕРИТЬ КОРРЕКТНОСТЬ ВВОДА С ПОМОЩЬЮ СООТВЕТСТВУЮЩЕЙ ФУНКЦИИ ВАЛИДАЦИИ
65         ЕСЛИ ВВОД КОРРЕКТЕН ТО:
66             ЕСЛИ ЭТО НЕ ПОСЛЕДНЕЕ ПОЛЕ В ЗАПИСИ ТО:
67                 ПЕРЕЙТИ К СЛЕДУЮЩЕМУ ПОЛЮ
68             ИНАЧЕ (ЭТО ПОСЛЕДНЕЕ ПОЛЕ):
69                 ЗАПИСАТЬ ВСЕ ПОЛЯ ЗАПИСИ В ФАЙЛ
70                 УВЕЛИЧИТЬ СЧЕТЧИК ЗАПИСЕЙ
71                 ВЫВЕСТИ СООБЩЕНИЕ О СОХРАНЕНИИ С ДЕТАЛЯМИ ЗАПИСИ
72                 СБРОСИТЬ НОМЕР ПОЛЯ НА НОЛЬ ДЛЯ НОВОЙ ЗАПИСИ
73             ИНАЧЕ:
74                 ВЫВЕСТИ СООБЩЕНИЕ О НЕВЕРНОМ ФОРМАТЕ С ПРИМЕРОМ ПРАВИЛЬНОГО ВВОДА
75         ЗАКРЫТЬ ФАЙЛ
76     ВЫВЕСТИ ИТОГОВУЮ ИНФОРМАЦИЮ: ИМЯ ФАЙЛА И КОЛИЧЕСТВО СОХРАНЕННЫХ ЗАПИСЕЙ
77
78 функция validate_filename(имя_файла):
79     ПРОВЕРИТЬ СООТВЕТСТВИЕ РЕГУЛЯРНОМУ ВЫРАЖЕНИЮ: латинские буквы, цифры, дефис,
    подчеркивание, расширение .txt
80     ВЕРНУТЬ РЕЗУЛЬТАТ ПРОВЕРКИ
81
82 функция validate_license(номерной_знак):
83     ПРОВЕРИТЬ СООТВЕТСТВИЕ РЕГУЛЯРНОМУ ВЫРАЖЕНИЮ: буква из набора [АВЕКМНОРСТУХ] + 3
    цифры + 2 буквы из того же набора + 2-3 цифры + RUS
84     ВЕРНУТЬ РЕЗУЛЬТАТ ПРОВЕРКИ
85
86 функция validate_brand(марка):
87     ПРОВЕРИТЬ СООТВЕТСТВИЕ РЕГУЛЯРНОМУ ВЫРАЖЕНИЮ: русские и английские буквы, цифры,
    пробел, дефис, подчеркивание, точка, восклицательный знак
88     ВЕРНУТЬ РЕЗУЛЬТАТ ПРОВЕРКИ
89
90 функция validate_model(модель):
91     ПРОВЕРИТЬ СООТВЕТСТВИЕ РЕГУЛЯРНОМУ ВЫРАЖЕНИЮ: русские и английские буквы, цифры,
    пробел, дефис, подчеркивание, точка, восклицательный знак, скобки
92     ВЕРНУТЬ РЕЗУЛЬТАТ ПРОВЕРКИ
93
94 функция validate_surname(фамилия):
95     ПРОВЕРИТЬ СООТВЕТСТВИЕ РЕГУЛЯРНОМУ ВЫРАЖЕНИЮ: только русские и английские буквы,
    дефис для двойных фамилий
96     ВЕРНУТЬ РЕЗУЛЬТАТ ПРОВЕРКИ
97
98 функция validate_address(адрес):
```

```
99    ПРОВЕРИТЬ СООТВЕТСТВИЕ РЕГУЛЯРНОМУ ВЫРАЖЕНИЮ: русские и английские буквы, цифры,  
      пробел, точка, запятая, дефис, подчеркивание  
100   ВЕРНУТЬ РЕЗУЛЬТАТ ПРОВЕРКИ  
101  
102   функция validate_release_year(год_выпуска):  
103     ЕСЛИ ГОД ПУСТОЙ ИЛИ СОДЕРЖИТ НЕЦИФРЫ ТО:  
104       ВЫВЕСТИ СООБЩЕНИЕ ОБ ОШИБКЕ  
105       ВЕРНУТЬ ЛОЖЬ  
106     ПРЕОБРАЗОВАТЬ СТРОКУ В ЧИСЛО  
107     ОПРЕДЕЛИТЬ ТЕКУЩИЙ ГОД СИСТЕМЫ  
108     ЕСЛИ ГОД ВНЕ ДИАПАЗОНА ОТ 1960 ДО ТЕКУЩЕГО ГОДА ТО:  
109       ВЫВЕСТИ СООБЩЕНИЕ О НЕДОПУСТИМОМ ДИАПАЗОНЕ  
110       ВЕРНУТЬ ЛОЖЬ  
111     ВЕРНУТЬ ИСТИНУ
```

3

Сведения о программной реализации

3.1 Информация об используемых языках программирования и их версиях, о версиях систем программирования

Языки программирования: C++17 (GNU C Compiler 15.2.1)

Утилита сборки программы: GNU Make 4.4.1

Системы программирования: Visual Studio Code 1.106.0

3.2 Описание входных и выходных данных

Входные данные:

Пользователь выбирает опции функционирования программы и записывает данные в выбранный файл посредством ввода текста. Ввод представляет собой номер выбранной опции – число, имя файла – текст, с которым будет связано дальнейшее выполнение программы, а также последовательный ввод полей для записи в файл – текст

Структура входных данных:

данные вводятся по одному на строку, таким образом пользователь может ввести только одну опцию за раз, а заполнение полей записей файла происходит по одному на строку (одно поле – одна строка)

Выходные данные и их структура:

Пользователь имеет два вида выходных данных:

1. Консольные сообщения: представляют собой текст, необходимый пользователю для пользования программой. Сообщения включают в себя: назначение программы, опции пользования программой, информацию об успешном/неуспешном открытии файла, предупреждения, если пользователь неправильно ввел поле записи, а также сообщение о завершении программы.
2. Файлы: представляют из себя текстовые файлы, содержащие записи, введенные пользователем в порядке записи по одному на строку, имя файла либо вводится пользователем, либо присваивается стандартное программой: `car_data.txt` или `license_data.txt`, в зависимости от назначения программы.

Файл автомобильных сведений:

состоит из записей, каждая из которых содержит три поля – марка, модель и номерной знак.

Файл регистрационных сведений:

состоит из записей, каждая из которых содержит четыре поля – номерной знак, фамилия, адрес владельца и год выпуска.

3.3 Краткое описание спроектированных программных единиц

файл main.cpp:

Функция	Назначение	Аргументы	Возвращает
main	Точка входа программы	Отсутствуют	Код завершения программы. Тип: int

файл file_handler.hpp:

Подключает следующие библиотеки:

Библиотека	Назначение
cstring	библиотека для работы со строками C (memset, strcpy)
cstdio	библиотека стандартного ввода/вывода C (printf, fopen, fgets)
cstdlib	стандартная библиотека C (EXIT_SUCCESS)
ctime	библиотека для работы со временем (time, localtime, получение текущего года)
cctype	библиотека для классификации символов (isdigit)
string	библиотека для работы с C++ строками (std::string)
regex	библиотека для работы с регулярными выражениями (std::regex_match)
filesystem	библиотека для работы с файловой системой (проверка существования файла)
stdio_ext.h	расширение стандартного ввода/вывода (__fpurge для очистки потока stdin)
locale	библиотека для установки локали (setlocale, поддержка кириллицы)

Содержит следующие структуры:

Структура	Назначение	Поля
CarData	Структура для временного хранения данных автомобиля перед записью в файл	brand – марка автомобиля (например: Toyota, BMW, Lada), model – модель автомобиля (например: Camry, X5, Vesta), license – номерной знак автомобиля (формат: A999AA99RUS или A999AA999RUS)
LicenseData	Структура для временного хранения регистрационных данных перед записью в файл	license – номерной знак автомобиля (связующее поле с автомобильными данными), surname – фамилия владельца ав-

Структура	Назначение	Поля
		томобилия (только буквы, дефис для двойных фамилий), address – адрес регистрации владельца (буквы, цифры, знаки препинания), release_year – год выпуска автомобиля (четыре цифры от 1960 до текущего года)

Также содержит все прототипы функций: (подробнее о функциях ниже)

Прототип функции
<code>bool validate_filename(const std::string& filename)</code>
<code>bool validate_license(const std::string& license)</code>
<code>bool validate_brand(const std::string& brand)</code>
<code>bool validate_model(const std::string& model)</code>
<code>bool validate_surname(const std::string& surname)</code>
<code>bool validate_address(const std::string& address)</code>
<code>bool validate_release_year(const std::string& release_year)</code>
<code>std::pair<std::string, std::string> file_info(char* buffer, const size_t& buf_size, const std::string& default_filename)</code>
<code>void write_data(char* buffer, const size_t& buf_size, const size_t& option)</code>

файл `file_handler.cpp`:

Подключает содержимое `file_handler.hpp` и содержит реализации функций

Под правильностью введенных полей, имен файлов, понимается:

1. в случае полей: это отсутствие *неподходящих символов* и *правильный* порядок элементов.
2. в случае имени файла: это *подходящее* имя файла, которое операционная система способна создать.

Функция	Назначение	Аргументы	Возвращает
<code>validate_filename</code>	Валидация имени файла	filename – имя файла. Тип: const std::string&	true если имя соответствует формату (латиница, цифры, дефис, подчеркивание, расширение .txt), иначе false. Тип: bool
<code>validate_license</code>	Валидация поля номерной знак	license – содержимое поля номерной знак. Тип: const std::string&	true если номерной знак соответствует формату A999AA99RUS или A999AA999RUS (где A из набора АВЕКМНОРСТУХ), иначе false. Тип: bool
<code>validate_brand</code>	Валидация поля марка автомобиля	brand – содержимое поля марка. Тип: const std::string&	true если марка содержит только разрешенные символы (буквы, цифры, пробел, дефис, подчеркивание, точка, восклицательный знак), иначе false. Тип: bool

Функция	Назначение	Аргументы	Возвращает
validate_model	Валидация поля модель автомобиля	model – содержимое поля модель. Тип: const std::string&	true если модель содержит только разрешенные символы (буквы, цифры, пробел, дефис, подчеркивание, точка, восклицательный знак, скобки), иначе false. Тип: bool
validate_surname	Валидация поля фамилия владельца	surname – содержимое поля фамилия. Тип: const std::string&	true если фамилия содержит только буквы (русские/английские) и дефис, иначе false. Тип: bool
validate_address	Валидация поля адрес владельца	address – содержимое поля адрес. Тип: const std::string&	true если адрес содержит только разрешенные символы (буквы, цифры, пробел, точка, запятая, дефис, подчеркивание), иначе false. Тип: bool
validate_release_year	Валидация поля год выпуска	release_year – содержимое поля год выпуска. Тип: const std::string&	true если год выпуска состоит из четырех цифр и находится в диапазоне от 1960 до текущего года включительно, иначе false. Тип: bool
file_info	Запрашивает у пользователя имя файла, проверяет его корректность, определяет режим открытия (добавление или перезапись) и возвращает эти данные	buffer – буфер для ввода, buf_size – размер буфера, default_filename – стандартное имя файла (car_data.txt или license_data.txt). Тип: char*, const size_t&, const std::string&	Пара значений: первое – имя файла, второе – режим открытия («w» для перезаписи, «a» для добавления). Если пользователь выбрал выход, имя файла возвращается пустым. Тип: std::pair<std::string, std::string>
write_data	Универсальная функция записи данных. В зависимости от переданной опции записывает либо автомобильные дан-	buffer – буфер для ввода, buf_size – размер буфера, option – тип запи-	Ничего. Тип: void

Функция	Назначение	Аргументы	Возвращает
	ные (option=1), либо регистрационные данные (option=2). Выполняет валидацию каждого поля и сохраняет записи в файл	сываемых данных (1 - автомобильные, 2 - регистрационные). Тип: char*, const size_t&, const size_t&	

3.4 Список диагностических сообщений с указанием возможных причин их появления

Сообщение	Возможная причина
Ошибка: введите 1, 2 или q	Пользователь ввел символ, отличный от „1“, „2“ или „q“ в главном меню.
Ошибка: неверное имя файла. Имя должно содержать только латинские буквы, цифры, дефис или подчеркивание и иметь расширение .txt	Пользователь ввел имя файла, содержащее недопустимые символы (например, кириллицу, пробелы, спецсимволы) или не имеющее расширения .txt.
Файл „%s“ уже существует. Enter - добавить записи в конец, N - перезаписать, q - вернуться в меню	Пользователь ввел имя файла, который уже существует в текущей директории.
Ошибка: не удалось открыть файл „%s“. Проверьте права доступа или путь к файлу	Файл не может быть открыт из-за отсутствия прав доступа, блокировки другим процессом или неверного пути.
Ошибка: поле не может быть пустым. Введите значение или „q“ для выхода	Пользователь нажал Enter, не введя никаких данных, когда требовалось заполнить поле.
Ошибка: неверный формат марки. Используйте только разрешенные символы: буквы, цифры, пробел, . _ ! -	Поле марка содержит недопустимые символы (например, @, #, \$, %) или пустое.
Ошибка: неверный формат модели. Используйте только разрешенные символы: буквы, цифры, пробел, . _ ! () -	Поле модель содержит недопустимые символы (например, @, #, \$, %) или пустое.
Ошибка: неверный формат номерного знака. Правильный формат: A999AA99RUS или A999AA999RUS. Где А - одна из букв: А, В, Е, К, М, Н, О, Р, С, Т, У, Х. Например: М976ММ777RUS	Пользователь ввел номерной знак, не соответствующий формату: неверная первая буква, неправильное количество цифр, отсутствует RUS в конце или использованы недопустимые символы.
Ошибка: неверный формат фамилии. Фамилия должна содержать только буквы (русские/английские). Для двойных фамилий используйте дефис	Поле фамилия содержит цифры, спецсимволы или пустое.
Ошибка: неверный формат адреса. Используйте только буквы, цифры, пробел, точку, запятую, дефис, подчеркивание	Поле адрес содержит недопустимые символы (например, @, #, \$, %, !).
Ошибка: год выпуска не может быть пустым	Пользователь не ввел год выпуска (нажал Enter).

Сообщение	Возможная причина
Ошибка: год должен состоять только из цифр (например: 2015)	Поле год выпуска содержит буквы или другие нецифровые символы.
Ошибка: год выпуска не может быть раньше 1960 года	Введенный год выпуска меньше минимально допустимого (1960).
Ошибка: год выпуска не может быть позже текущего (20XX)	Введенный год выпуска превышает текущий год системы.
Возврат в главное меню (текущая запись не сохранена)	Пользователь ввел „menu“ во время ввода данных, прервав заполнение текущей записи.
Ввод завершен. Сохраняем данные...	Пользователь ввел „q“ для завершения сеанса ввода данных.
Нажмите Enter...	Программа ожидает подтверждения пользователя перед возвратом в главное меню.
Программа завершена	Пользователь выбрал выход из программы или достигнут конец ввода.

4

Инструкция использования программы

5

Инструкция использования программы

Инструкция полностью соответствует указаниям на экране.

Но все же рассмотрим как пользоваться программой:

Для запуска программы пользователь должен запустить файл: `./file_h`

Затем пользователь увидит следующее:

ПРОГРАММА ДЛЯ СОЗДАНИЯ ФАЙЛОВ	
ГЛАВНОЕ МЕНЮ	
1) Автомобильные данные	
2) Регистрационные данные	
q) Выход	
Выбор:	

Теперь у пользователя есть выбор, как взаимодействовать с программой.
Рассмотрим основные варианты:

Допустим пользователь ввел 1. Теперь он увидит следующее:

АВТОМОБИЛЬНЫЕ ДАННЫЕ	
* „q“ - выход из ввода	
Введите название файла (Enter=„car_data.txt“, „q“=меню):	

Программа ожидает от пользователя название файла, в который будут записаны сведения. Если пользователь нажмет Enter, то он будет работать с файлом car_data.txt. Если пользователь введет любое другое имя файла, которое удовлетворяет требованиям, например file1.txt, он увидит следующие сообщения:

Будет создан новый файл „file1.txt“	
Файл „file1.txt“ успешно открыт	
=====	
ВВОД ДАННЫХ	
=====	
* „q“ - завершить ввод и сохранить файл, а затем вернуться в меню	
=====	
— Поле 1/3: МАРКА автомобиля —	
Разрешены: русские/английские буквы, цифры, пробел, дефис(-), подчеркивание(_), точка(.), воскл.знак(!)	
>	

Но если файл file1.txt уже существует будет выведено следующее сообщение:

Файл „file1.txt“ уже существует.
Enter - добавить записи в конец
N - перезаписать файл с начала
q - вернуться в главное меню
Ваш выбор:

В таком случае программа в зависимости от ввода пользователя будет либо дописывать сведения в файл, либо заполнит его с нуля, либо вернется в меню.

Теперь программа ожидает от пользователя заполнение поля марка.

Если пользователь заполнит его без ошибок, то он перейдет к записи следующего поля. Если же пользователь совершит ошибку при вводе, то он увидит следующее сообщение:

Ошибка: неверный формат марки.
Используйте только разрешенные символы: буквы, цифры, пробел, . ! -
Попробуйте еще раз или введите „q“ для выхода.

— Поле 1/3: МАРКА автомобиля —
Разрешены: русские/английские буквы, цифры, пробел, дефис(-), подчеркивание(_), точка(.)
>

Это сообщение будет повторяться до тех пор, пока пользователь не введет поле правильно, или не решит завершить ввод, нажав q, или выйти в меню, набрав menu.

Как только пользователь правильно введет поле, он перейдет к заполнению следующего:

Поле принято

— Поле 2/3: МОДЕЛЬ автомобиля —
Разрешены: русские/английские буквы, цифры, пробел, дефис(-), подчеркивание(_), точка(.) скобки()
>

Далее поле номерного знака:

— Поле 3/3: НОМЕРНОЙ ЗНАК —
Формат: A999AA99RUS или A999AA999RUS (где A - одна из букв: A, B, E, K, M, H, O, P, C, T, Y, X) или дефисы и пробелы
>

Стоит отметить, что данные записываются в файл при условии, что все поля были введены верно.

Как только пользователь заполнит первую запись он увидит сообщение:

ЗАПИСЬ 1 СОХРАНЕНА В ФАЙЛ

Марка: Toyota

Модель: Supra

Номер: M976MM777RUS

— Поле 1/3: МАРКА автомобиля —

Разрешены: русские/английские буквы, цифры, пробел, дефис(-), подчеркивание(_), точка(.)

>

Если пользователь в процессе заполнения введет q, он увидит следующее сообщение:

Ввод завершен. Сохраняем данные...

РАБОТА ЗАВЕРШЕНА

Файл: car_data.txt

Сохранено записей: 1

Если пользователь введет menu во время заполнения, он увидит:

Возврат в главное меню (текущая запись не сохранена)

Затем программа возвращается в главное меню:

Нажмите Enter...

После нажатия Enter программа снова отображает главное меню, позволяя выбрать новый режим работы или выйти.

Если пользователь в главном меню введет q, он увидит:

Программа завершена.

6

Листинг программной разработки

файл main.cpp:

```
1 #include "file_handler.hpp"
2 #include <locale>
3
4 int main() {
5     setlocale(LC_ALL, "ru_RU.UTF-8");
6
7     printf("=====\\n");
8     printf("    ПРОГРАММА ДЛЯ СОЗДАНИЯ ФАЙЛОВ    \\n");
9     printf("=====\\n\\n");
10
11     const size_t BUF_SIZE = 512;
12     char BUFFER[BUF_SIZE];
13
14     printf("\\n");
15     printf("    \\n");
16     printf("    ГЛАВНОЕ МЕНЮ    \\n");
17     printf("    \\n");
18     printf("    1) Автомобильные данные    \\n");
19     printf("    2) Регистрационные данные    \\n");
20     printf("    q) Выход    \\n");
21     printf("    \\n");
22     printf("Выбор: ");
23     fflush(stdout);
24
25     while (fgets(BUFFER, BUF_SIZE, stdin) != nullptr) {
26         __fpurge(stdin);
27         BUFFER[strcspn(BUFFER, "\\n")] = 0;
28
29         if (BUFFER[0] == 'q' || BUFFER[0] == 'Q') {
30             break;
31         }
32         else if (BUFFER[0] == '1' || BUFFER[0] == '2') {
33             printf("=====\\n");
34             printf("    %s\\n", BUFFER[0] == '1' ? "АВТОМОБИЛЬНЫЕ ДАННЫЕ" :
"РЕГИСТРАЦИОННЫЕ ДАННЫЕ");
35             printf("=====\\n");
36             printf("* 'q' - возвращение в меню\\n");
37             fflush(stdout);
38
```

```

39     write_data(BUFFER, BUF_SIZE, BUFFER[0] - '0');
40     __fpurge(stdin);
41
42     printf("\nНажмите Enter...");
43     fflush(stdout);
44     getchar();
45     __fpurge(stdin);
46 }
47 else {
48     printf("Ошибка: введите 1, 2 или q\n");
49     fflush(stdout);
50 }
51
52 printf("\n");
53 printf("
54 printf("
55 printf("
56 printf("
57 printf("
58 printf("
59 printf("
60 printf("Выбор: ");
61 fflush(stdout);
62 }
63
64 printf("\nПрограмма завершена.\n");
65 fflush(stdout);
66 return 0;
67 }

```

ГЛАВНОЕ МЕНЮ	
1) Автомобильные данные	
2) Регистрационные данные	
q) Выход	

файл file_handler.hpp

```

1 #ifndef FILE_HANDLER_HPP
2 #define FILE_HANDLER_HPP
3
4 #include <cstring>
5 #include <cstdio>
6 #include <cstdlib>
7 #include <locale>
8 #include <filesystem>
9 #include <regex>
10 #include <string>
11 #include <ctime>
12 #include <cctype>
13 #include <stdio_ext.h>
14
15 struct CarData {
16     std::string license;
17     std::string brand;
18     std::string model;
19 };
20
21 struct LicenseData {
22     std::string license;
23     std::string surname;
24     std::string address;
25     std::string release_year;
26 };
27
28 bool validate_filename(const std::string& filename);
29 bool validate_license(const std::string& license);
30 bool validate_brand(const std::string& brand);
31 bool validate_model(const std::string& model);

```

```

32 bool validate_surname(const std::string& surname);
33 bool validate_address(const std::string& address);
34 bool validate_release_year(const std::string& release_year);
35
36 std::pair<std::string, std::string> file_info(
37     char* buffer,
38     const size_t& buf_size,
39     const std::string& default_filename
40 );
41 void write_data(char* buffer, const size_t& buf_size, const size_t& option);
42
43 #endif

```

файл file_handler.cpp:

```

1  #include "file_handler.hpp"
2
3  const std::string DEFAULT_FILENAME_1 = "car_data.txt";
4  const std::string DEFAULT_FILENAME_2 = "license_data.txt";
5
6  bool validate_filename(const std::string& filename) {
7      return std::regex_match(filename, std::regex("[a-zA-Z0-9_-]+\\.txt$"));
8  }
9
10 bool validate_license(const std::string& license) {
11     bool cond1 = std::regex_match(license, std::regex("[АБЕКМНОРСТУХ][0-9]{3}"
12     [АБЕКМНОРСТУХ]{2}[0-9]{2,3}RUS$"));
13     bool cond2 = std::regex_match(license, std::regex("[- ]+$"));
14     return cond1 || cond2;
15 }
16
17 bool validate_brand(const std::string& brand) {
18     return std::regex_match(brand, std::regex("[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-
19     Za-z0-9-_. -]+$"));
20 }
21
22 bool validate_model(const std::string& model) {
23     return std::regex_match(model, std::regex("[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-яА-
24     Za-z0-9-_.() -]+$"));
25 }
26
27 bool validate_surname(const std::string& surname) {
28     return std::regex_match(surname, std::regex("[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-
29     яА-Za-z -]+$"));
30 }
31
32 bool validate_address(const std::string& address) {
33     return std::regex_match(address, std::regex("[ФфРрТтУуХхЦцЧчШшЩщЪъЬьЭэЮюЁёЫыА-Яа-
34     яА-Za-z0-9_., -]+$"));
35 }
36
37 bool validate_release_year(const std::string& release_year) {
38     if (std::regex_match(release_year, std::regex("[- ]+$"))) {
39         return true;
40     }
41     static size_t lower_bound_year = 1960;
42     std::time_t t = std::time(nullptr);
43     std::tm *const pTInfo = std::localtime(&t);
44     static size_t current_year = 1900 + pTInfo->tm_year;
45
46     if (release_year.empty()) {
47         printf("Ошибка: год выпуска не может быть пустым.\n");
48         return false;
49     }

```

```

43     }
44
45     // Проверка длины строки (год не может быть длиннее 4 цифр)
46     if (release_year.length() > 4) {
47         printf("Ошибка: год должен состоять из 4 цифр (например: 2015).\n");
48         return false;
49     }
50
51     for (char c : release_year) {
52         if (!isdigit(c)) {
53             printf("Ошибка: год должен состоять только из цифр (например: 2015).\n");
54             return false;
55         }
56     }
57
58     // Безопасное преобразование с обработкой исключения
59     int year = 0;
60     try {
61         year = std::stoi(release_year);
62     } catch (const std::out_of_range&) {
63         printf("Ошибка: введенное значение слишком большое.\n");
64         return false;
65     } catch (const std::invalid_argument&) {
66         printf("Ошибка: неверный формат года.\n");
67         return false;
68     }
69
70     if (year < lower_bound_year) {
71         printf("Ошибка: год выпуска не может быть раньше %zu года.\n",
lower_bound_year);
72         return false;
73     }
74     if (year > current_year) {
75         printf("Ошибка: год выпуска не может быть позже текущего (%zu).\n",
current_year);
76         return false;
77     }
78
79     return true;
80 }
81 std::pair<std::string, std::string> file_info(
82     char* BUFFER,
83     const size_t& BUF_SIZE,
84     const std::string& default_file_name) {
85
86     std::pair<std::string, std::string> result;
87     std::string file_name;
88
89     printf("Введите название файла (Enter='%s', 'q'=меню): ",
default_file_name.c_str());
90     fflush(stdout);
91
92     while (fgets(BUFFER, BUF_SIZE, stdin) != nullptr) {
93         __fpurge(stdin);
94         BUFFER[strcspn(BUFFER, "\n")] = 0;
95         file_name = BUFFER;
96
97         if (file_name == "q" || file_name == "Q") {
98             result.first = "";
99             result.second = "";
100             return result;

```

```

101     }
102
103     if (file_name.empty()) {
104         file_name = default_file_name;
105         break;
106     }
107
108     if (validate_filename(file_name)) {
109         break;
110     } else {
111         printf("Ошибка: неверное имя файла.\n");
112         printf("Имя должно содержать только латинские буквы, цифры, дефис или
подчеркивание\n");
113         printf("и иметь расширение .txt (например: my_data.txt или
car-2024.txt)\n\n");
114         printf("Введите название файла (Enter='%s', 'q'=меню): ",
default_file_name.c_str());
115         fflush(stdout);
116     }
117 }
118
119 result.first = file_name;
120
121 if (std::filesystem::exists(file_name)) {
122     printf("\nФайл '%s' уже существует.\n", file_name.c_str());
123     printf("Enter - добавить записи в конец\n");
124     printf("N - перезаписать файл с начала\n");
125     printf("q - вернуться в главное меню\n");
126     printf("Ваш выбор: ");
127     fflush(stdout);
128
129     if (fgets(BUFFER, BUF_SIZE, stdin) != nullptr) {
130         __fpurge(stdin);
131         if (BUFFER[0] == 'q' || BUFFER[0] == 'Q') {
132             result.first = "";
133             result.second = "";
134             return result;
135         }
136
137         if (BUFFER[0] == 'n' || BUFFER[0] == 'N') {
138             result.second = "w";
139             printf("Режим: перезапись файла\n");
140         } else {
141             result.second = "a";
142             printf("Режим: добавление в конец файла\n");
143         }
144     } else {
145         result.second = "a";
146     }
147     fflush(stdout);
148 } else {
149     result.second = "w";
150     printf("Будет создан новый файл '%s'\n", file_name.c_str());
151     fflush(stdout);
152 }
153
154 return result;
155 }
156
157 void write_data(char* BUFFER, const size_t& BUF_SIZE, const size_t& option) {
158     std::pair<std::string, std::string> file_information;
159

```

```

160     if (option == 1)
161         file_information = file_info(BUFFER, BUF_SIZE, DEFAULT_FILENAME_1);
162     else
163         file_information = file_info(BUFFER, BUF_SIZE, DEFAULT_FILENAME_2);
164
165     if (file_information.first.empty()) {
166         printf("Возврат в главное меню\n");
167         fflush(stdout);
168         return;
169     }
170
171     FILE* file = std::fopen(file_information.first.c_str(),
172                             file_information.second.c_str());
173     if (!file) {
174         printf("Ошибка: не удалось открыть файл '%s'\n",
175               file_information.first.c_str());
176         printf("Проверьте права доступа или путь к файлу.\n");
177         fflush(stdout);
178         return;
179     }
180     printf("\nФайл '%s' успешно открыт\n", file_information.first.c_str());
181     printf("===== \n");
182     printf("          ВВОД ДАННЫХ\n");
183     printf("===== \n");
184     printf("* 'q' - завершить ввод и сохранить файл, а затем вернуться в меню\n");
185     printf("===== \n");
186     fflush(stdout);
187
188     int field_index = 0;
189     int note_number = 0;
190
191     const std::string field_names_1[3] = {
192         "МАРКА автомобиля",
193         "МОДЕЛЬ автомобиля",
194         "НОМЕРНОЙ ЗНАК"
195     };
196
197     const std::string field_hints_1[3] = {
198         "Разрешены: русские/английские буквы, цифры, пробел, дефис(-),",
199         "подчеркивание(_), точка(.)",
200         "Разрешены: русские/английские буквы, цифры, пробел, дефис(-),",
201         "подчеркивание(_), точка(.), скобки()",
202         "Формат: A999AA999RUS или A999AA999RUS (где A - одна из букв: A, B, E, K, M, H,",
203         "O, P, C, T, Y, X) или дефисы и пробелы"
204     };
205
206     const std::string field_names_2[4] = {
207         "НОМЕРНОЙ ЗНАК",
208         "ФАМИЛИЯ владельца",
209         "АДРЕС владельца",
210         "ГОД ВЫПУСКА"
211     };
212
213     const std::string field_hints_2[4] = {
214         "Формат: A999AA999RUS или A999AA999RUS (где A - одна из букв: A, B, E, K, M, H,",
215         "O, P, C, T, Y, X)",
216         "Разрешены: буквы (русские/английские), дефис для двойных фамилий и пробел",
217         "Разрешены: буквы, цифры, пробел, точка(.), запятая(,), дефис(-),",
218         "подчеркивание(_)",
219         "Четыре цифры от 1960 до текущего года, а также дефис(-) и пробел"
220     };

```



```

215
216     CarData temp_1;
217     LicenseData temp_2;
218
219     if (option == 1) {
220         printf("\n--- Поле %d/3: %s ---\n", field_index + 1,
221             field_names_1[field_index].c_str());
222         printf("    %s\n", field_hints_1[field_index].c_str());
223     } else {
224         printf("\n--- Поле %d/4: %s ---\n", field_index + 1,
225             field_names_2[field_index].c_str());
226         printf("    %s\n", field_hints_2[field_index].c_str());
227     }
228     printf("> ");
229     fflush(stdout);
230
231     while (fgets(BUFFER, BUF_SIZE, stdin) != nullptr) {
232         __fpurge(stdin);
233         BUFFER[strcspn(BUFFER, "\n")] = 0;
234         std::string input = BUFFER;
235
236         if (input == "q" || input == "Q") {
237             printf("\nВвод завершен. Сохраняем данные...\n");
238             break;
239         }
240
241         if (input == "menu" || input == "MENU") {
242             fclose(file);
243             printf("Возврат в главное меню (текущая запись не сохранена)\n");
244             fflush(stdout);
245             return;
246         }
247
248         if (input.empty()) {
249             printf("Ошибка: поле не может быть пустым. Введите значение или 'q' для
250             выхода.\n");
251         } else {
252             bool valid = false;
253
254             if (option == 1) {
255                 switch (field_index) {
256                     case 0: // МАРКА
257                         temp_1.brand = input;
258                         if (validate_brand(temp_1.brand)) {
259                             field_index = 1;
260                             valid = true;
261                             printf("Поле принято\n");
262                         } else {
263                             printf("Ошибка: неверный формат марки.\n");
264                             printf("Используйте только разрешенные символы: буквы,
265                             цифры, пробел, . _ ! -\n");
266                         }
267                     break;
268                     case 1: // МОДЕЛЬ
269                         temp_1.model = input;
270                         if (validate_model(temp_1.model)) {
271                             field_index = 2;
272                             valid = true;
273                             printf("Поле принято\n");
274                         } else {
275                             printf("Ошибка: неверный формат модели.\n");

```

```

272             printf("Используйте только разрешенные символы: буквы,
цифры, пробел, . _ ! ( ) -\n");
273         }
274         break;
275     case 2: // НОМЕРНОЙ ЗНАК
276         temp_1.license = input;
277         if (validate_license(temp_1.license)) {
278             fprintf(file, "%s %s %s\n",
279                 temp_1.brand.c_str(),
280                 temp_1.model.c_str(),
281                 temp_1.license.c_str());
282             note_number++;
283             printf("\nЗАПИСЬ #%d СОХРАНЕНА В ФАЙЛ\n", note_number);
284             printf("Марка: %s\n", temp_1.brand.c_str());
285             printf("Модель: %s\n", temp_1.model.c_str());
286             printf("Номер: %s\n", temp_1.license.c_str());
287             printf("-----\n");
288             fflush(stdout);
289             field_index = 0;
290             temp_1 = CarData();
291             valid = true;
292         } else {
293             printf("Ошибка: неверный формат номерного знака.\n");
294             printf("Правильный формат: A999AA999RUS или
A999AA999RUS\n");
295             printf("Где А - одна из букв: А, В, Е, К, М, Н, О, Р, С, Т,
У, Х\n");
296             printf("Например: M976MM777RUS или A123BC98RUS\n");
297         }
298         break;
299     }
300     } else {
301         switch (field_index) {
302             case 0:
303                 temp_2.license = input;
304                 if (validate_license(temp_2.license)) {
305                     field_index = 1;
306                     valid = true;
307                     printf("Поле принято\n");
308                 } else {
309                     printf("Ошибка: неверный формат номерного знака.\n");
310                     printf("Правильный формат: A999AA999RUS или
A999AA999RUS\n");
311                     printf("Где А - одна из букв: А, В, Е, К, М, Н, О, Р, С, Т,
У, Х\n");
312                     printf("Например: M976MM777RUS или A123BC98RUS\n");
313                 }
314                 break;
315             case 1:
316                 temp_2.surname = input;
317                 if (validate_surname(temp_2.surname)) {
318                     field_index = 2;
319                     valid = true;
320                     printf("Поле принято\n");
321                 } else {
322                     printf("Ошибка: неверный формат фамилии.\n");
323                     printf("Фамилия должна содержать только буквы (русские/
английские).\n");
324                     printf("Для двойных фамилий используйте дефис (например:
Иванов-Петров).\n");
325                 }
326                 break;

```

```

327         case 2:
328             temp_2.address = input;
329             if (validate_address(temp_2.address)) {
330                 field_index = 3;
331                 valid = true;
332                 printf("Поле принято\n");
333             } else {
334                 printf("Ошибка: неверный формат адреса.\n");
335                 printf("Используйте только буквы, цифры, пробел, точку,
запятую, дефис, подчеркивание.\n");
336             }
337             break;
338         case 3:
339             temp_2.release_year = input;
340             if (validate_release_year(temp_2.release_year)) {
341                 fprintf(file, "%s %s %s %s\n",
342                     temp_2.license.c_str(),
343                     temp_2.surname.c_str(),
344                     temp_2.address.c_str(),
345                     temp_2.release_year.c_str());
346                 note_number++;
347                 printf("\nЗАПИСЬ #%d СОХРАНЕНА В ФАЙЛ\n", note_number);
348                 printf("Номер: %s\n", temp_2.license.c_str());
349                 printf("Владелец: %s\n", temp_2.surname.c_str());
350                 printf("Адрес: %s\n", temp_2.address.c_str());
351                 printf("Год выпуска: %s\n", temp_2.release_year.c_str());
352                 printf("=====\n");
353                 fflush(stdout);
354                 field_index = 0;
355                 temp_2 = LicenseData();
356                 valid = true;
357             }
358             break;
359     }
360 }
361
362 if (!valid && field_index != 3) {
363     printf("Попробуйте еще раз или введите 'q' для выхода.\n");
364     fflush(stdout);
365 }
366
367 if (option == 1) {
368     printf("\n--- Поле %d/3: %s ---\n", field_index + 1,
369         field_names_1[field_index].c_str());
370     printf("    %s\n", field_hints_1[field_index].c_str());
371 } else {
372     printf("\n--- Поле %d/4: %s ---\n", field_index + 1,
373         field_names_2[field_index].c_str());
374     printf("    %s\n", field_hints_2[field_index].c_str());
375 }
376 printf("> ");
377 fflush(stdout);
378
379 fclose(file);
380 printf("\n=====\n");
381 printf("РАБОТА ЗАВЕРШЕНА\n");
382 printf("Файл: %s\n", file_information.first.c_str());
383 printf("Сохранено записей: %d\n", note_number);
384 printf("=====\n");
385 fflush(stdout);

```

7

Результаты тестирования

Для демонстрации работы программы заполним файл car_data.txt:

Тест 1. Заполнение автомобильных данных

Пользователь выбирает опцию 1 (Автомобильные данные).

Программа запрашивает имя файла. Пользователь нажимает Enter, выбирая файл по умолчанию car_data.txt.

Заполнение первой записи:

- Марка: Toyota
- Модель: Camry
- Номерной знак: A123BC77RUS

Заполнение второй записи:

- Марка: BMW
- Модель: X5
- Номерной знак: B456KM99RUS

После ввода каждой записи программа выводит подтверждение сохранения.

Пользователь вводит „q“ для завершения.

Тест 2. Заполнение еще одной записи

Пользователь снова запускает программу и выбирает опцию 1.

При запросе имени файла нажимает Enter, выбирая существующий файл car_data.txt.

Программа сообщает, что файл уже существует, и предлагает выбор:

- Enter — добавить записи в конец
- N — перезаписать файл
- q — вернуться в меню

Пользователь выбирает Enter (добавление в конец).

Заполнение третьей записи:

- Марка: Lada
- Модель: Vesta
- Номерной знак: C789OP123RUS

После завершения ввода программа выводит итоговую информацию:

«Файл: car_data.txt, Сохранено записей: 3»

Тест 3. Просмотр содержимого car_data.txt

После завершения работы программы содержимое файла car_data.txt:

```
1 Toyota Camry A123BC77RUS
2 BMW X5 B456KM99RUS
3 Lada Vesta C789OP123RUS
```

Информация успешно записана в файл без каких-либо проблем.

Тест 4. Заполнение регистрационных данных

Пользователь выбирает опцию 2 (Регистрационные данные).

Вводит имя файла license_data.txt (или нажимает Enter для использования имени по умолчанию).

Заполнение первой записи:

- Номерной знак: A123BC77RUS
- Фамилия владельца: Иванов
- Адрес: г. Москва, ул. Тверская, д.10
- Год выпуска: 2020

Заполнение второй записи:

- Номерной знак: B456KM99RUS
- Фамилия владельца: Петров
- Адрес: г. Санкт-Петербург, Невский пр., д.25
- Год выпуска: 2018

После завершения ввода программа сохраняет обе записи.

Тест 5. Проверка обработки ошибок при вводе

При заполнении поля фамилия пользователь допустил ошибку:

Вместо «Сидоров» ввел «Сидоров123».

Программа вывела сообщение:

«Ошибка: неверный формат фамилии. Фамилия должна содержать только буквы (русские/английские). Для двойных фамилий используйте дефис.»

Пользователь исправил ввод на «Сидоров» и продолжил заполнение.

Заполнение третьей записи:

- Номерной знак: C789OP123RUS
- Фамилия владельца: Сидоров
- Адрес: г. Казань, ул. Баумана, д.5
- Год выпуска: 2022

Тест 6. Проверка обработки ошибок при вводе года

При попытке ввести некорректный год выпуска:

Ввод «1950»

Программа: «Ошибка: год выпуска не может быть раньше 1960 года»

Ввод «2025» (при условии, что текущий год 2024)

Программа: «Ошибка: год выпуска не может быть позже текущего (2024)»

Ввод «abcd»

Программа: «Ошибка: год должен состоять только из цифр (например: 2015)»

После корректировки ввода запись успешно сохраняется.

Тест 7. Дозапись в существующий файл license_data.txt

Пользователь снова запускает программу, выбирает опцию 2 и нажимает Enter при запросе имени файла.

Программа сообщает, что файл license_data.txt уже существует.

Пользователь выбирает Enter (добавление в конец).

Добавление четвертой записи:

- Номерной знак: D321XY77RUS
- Фамилия владельца: Кузнецов
- Адрес: г. Екатеринбург, ул. Ленина, д.15
- Год выпуска: 2021

Запись успешно добавлена в конец файла.

Тест 8. Просмотр содержимого license_data.txt

После завершения работы программы содержимое файла license_data.txt:

- 1 A123BC77RUS Иванов г. Москва, ул. Тверская, д.10 2020
- 2 B456KM99RUS Петров г. Санкт-Петербург, Невский пр., д.25 2018
- 3 C7890P123RUS Сидоров г. Казань, ул. Баумана, д.5 2022
- 4 D321XY77RUS Кузнецов г. Екатеринбург, ул. Ленина, д.15 2021

Информация успешно записана и дописана в файл.

Тест 9. Проверка выхода из программы

В главном меню пользователь вводит „q“.

Программа выводит сообщение «Программа завершена.» и завершает свою работу.

Проверка выхода через „menu“ во время ввода:

При заполнении полей пользователь вводит „menu“.

Программа выводит «Возврат в главное меню (текущая запись не сохранена)» и возвращается в главное меню без сохранения текущей записи.

Вывод

В результате тестирования можно отметить, что программа функционирует корректно:

- Все поля проходят валидацию в соответствии с заданными форматами
- Программа корректно обрабатывает ошибки ввода и выводит понятные подсказки
- Запись в файл (как создание нового, так и добавление в существующий) работает корректно
- Поддержка кириллицы в полях марка, модель, фамилия, адрес работает без ошибок
- Возможность выхода из программы и возврата в меню работает корректно